

Retrieval Stack Audit And Customization Roadmap

Project: RAG_ENTERPRISE_STARTER

Scope: Backend retrieval, chunking, embeddings, routing, retrieval modes, reranking, metadata filtering, answer grounding, and evaluation.

Purpose: Document the exact retrieval techniques implemented today, identify limitations relative to enterprise-grade RAG systems, and recommend which levers should stay as-is, become configurable, or be added later.

1. Executive Summary

This repository is already more than a basic RAG demo. It supports document-type-specific parsing and chunking, one dense embedding path, multiple retrieval modes, heuristic query routing, optional cross-encoder reranking, graph and temporal enrichment layers, a deep-research retrieval path, and several offline evaluation harnesses. It is best described as a strong PoC-grade retrieval system rather than an enterprise-final retrieval platform.

The main retrieval baseline today is hybrid search with linear score fusion, not Reciprocal Rank Fusion. The hybrid score is computed as $\text{combined_score} = \alpha * \text{vector_score} + (1 - \alpha) * \text{keyword_score}$ where $\alpha = 0.65$. That makes vector relevance dominant by default. This is a reasonable baseline, but it is not as robust as rank-based fusion when score distributions differ or lexical exact-match queries matter.

Several valuable levers are already present: custom query text, exact phrase bias, anchor-term extraction, source and locator filters, deep-research anchor scans, optional reranking, graph boosts, and temporal boosts. However, some enterprise levers are absent: no semantic cache, no full query rewriting or HyDE, no field-weighted keyword indexing, no diversity stage such as MMR, and no real user-query mining loop for evaluation.

2. Current Retrieval Pipeline

The runtime path is: parse source by document type, chunk source by document type, embed chunks with one dense model, route the query to a mode when mode is omitted, retrieve candidates, optionally rerank them, generate a grounded answer with citation validation, and evaluate behavior with fixture-based benchmark harnesses.

Main code surfaces: chunking in `backend/app/ingestion/chunking.py`, embedding in `backend/app/embedding/embedder.py`, retrieval orchestration in `backend/app/core_rag/retrieval.py`, query routing in `backend/app/core_rag/query_router.py`, DB search in `backend/app/db/repo_search.py`, reranking in `backend/app/core_rag/reranker.py`, answer grounding in `backend/app/core_rag/answering.py`, and evaluation in `backend/app/eval/*.py`.

3. Parsing And Chunking

Chunking is document-type specific. That is a meaningful strength of the current codebase because the repo does not force PDF pages, DOCX sections, spreadsheets, slides, and emails through a single chunking strategy.

3.1 Global chunk constants

Global constants in `chunking.py` are shared across multiple chunkers: `MIN_WORDS = 120`, `TARGET_WORDS = 320`, `OVERLAP_WORDS = 40`, and the PDF-only bridge constant `PDF_CROSS_PAGE_BRIDGE_WORDS = 140`. The default text windowing logic is a simple sliding word window of 320 words with 40-word overlap. If a part is shorter than the target size, it becomes a single chunk.

Chunk windows are heuristic, word-count based, and global. They are not tokenizer-aware, not semantic-boundary aware, and not query-adaptive.

3.2 Document-type behavior

PDF: each page is parsed as a part, then chunked per page. In addition, adjacent pages are stitched into cross-page bridge chunks using the tail of the previous page and the head of the next page, each with a 140-word window. This helps where meaning breaks across page boundaries.

DOCX: headings are treated as section markers. Paragraphs under a heading are buffered together, tables are handled separately, and chunks inherit section heading context. This is better than flat paragraph chunking for narrative and policy-style documents.

PPTX: each slide becomes a part, and speaker notes are appended when present. This preserves slide-level structure and allows retrieval over notes without inventing a separate storage model.

XLSX: each sheet becomes a part, rows are rendered into text lines such as Row N, and chunks are formed as row-range groups. This preserves enough spreadsheet structure for retrieval without a full table-aware semantic model.

EML: headers and body are parsed into separate parts. HTML bodies are converted to text if plain text is unavailable. Attachments are collected as metadata but not semantically indexed as part of the email body.

TXT and MD: TXT becomes one body block; Markdown preserves heading sections and heading levels so chunking remains section-aware even though markdown syntax is otherwise treated as text.

Limitations and consequences

Chunking is stronger than a naive text splitter, but still heuristic. There is no tokenizer-aware size policy, no learned chunking, no adaptive chunking by query intent, and no specialized chunking for code, contracts, dense tables, or diagram-heavy documents. Field-aware and layout-aware enterprise retrieval would need more than the current word-window logic.

4. Embedding Stack

The active embedding model is BAAI/bge-small-en-v1.5 from SentenceTransformers, configured in backend/app/core/config.py and loaded in backend/app/embedding/embedder.py. Embeddings are normalized at encode time with `normalize_embeddings=True`. Batch size defaults to 32. The vector dimension is inferred from the model, with a fallback dummy encode if needed.

This means the repository uses one global dense embedding space for all source types and all query types. There is no sparse encoder, no dual-index learned hybrid setup, no field-specific embedding path, and no multi-vector chunk representation.

Limitations and consequences

Dense-only embedding is a solid PoC baseline, but it will underperform on exact identifiers, codes, SKUs, dates, and some structured or multilingual retrieval patterns unless lexical retrieval compensates. It also makes it harder to optimize independently for titles, table cells, or relationships without adding more indexing layers.

5. Query Routing And Retrieval Modes

The router only applies when mode is omitted. If a caller explicitly asks for a mode, that mode is respected. The router uses heuristic intent detection to choose among vector, keyword, hybrid, graph_hybrid, and full. Deep lookup is a separate explicit retrieval surface.

Current retrieval modes are: vector, keyword, hybrid, graph_hybrid, full, and retrieval-only deep_lookup. The default retrieval mode is hybrid, and `USE_QUERY_ROUTER=true` by default.

Temporal-looking queries route to full if temporal artifacts are ready, otherwise they fall back to hybrid. Relationship-looking queries route to graph_hybrid if graph artifacts are ready, otherwise they fall back to hybrid. Exact-lookup-like queries route to keyword. Everything else defaults to hybrid.

The current exact-lookup detector is useful but narrow. It looks for signals like quotation marks, find, mention, contains, exact, locate, or where does. It does not yet explicitly detect quote lookup phrases such as final words, last sentence, exact wording, or code and SKU patterns.

Limitations and consequences

This router is deterministic and interpretable, which is good for a PoC. But it is not confidence-aware and does not yet use retrieval failures or answer failures to adapt the strategy. Enterprise routing usually evolves toward richer lexical-pattern detection, confidence thresholds, and sometimes lightweight model-based classification.

6. Exact Retrieval Techniques And Formulas

Vector retrieval uses PGVector nearest-neighbor search over chunk embeddings. The vector score exposed downstream is computed as $\max(0.0, 1.0 - \text{distance})$. Keyword retrieval uses PostgreSQL full-text search with `websearch_to_tsquery`, falls back to `plainto_tsquery` if needed, and then falls back again to a soft OR-anchor query when no rows are found.

Keyword scores are normalized in `retrieval.py` using $\text{keyword_score} = \text{rank_score} / \text{max_rank}$. Hybrid mode merges vector and keyword candidates by chunk id, then computes $\text{combined_score} = \alpha * \text{vector_score} + (1.0 - \alpha) * \text{keyword_score}$ with `HYBRID_ALPHA = 0.65`. This is weighted linear score fusion, not Reciprocal Rank Fusion.

Candidate caps are globally configured. Defaults are `VECTOR_CANDIDATES = 30`, `KEYWORD_CANDIDATES = 30`, and `TOP_K_INITIAL = 30`. This means recall is bounded before any answer generation step. Hybrid is therefore not a strict superset of keyword in practice, because lexical hits can be lost during score fusion, candidate truncation, reranking, or answer grounding.

Limitations and consequences

Linear score fusion is easy to reason about, but it is brittle when score distributions differ or when one retriever finds a crucial exact match that the other does not. Enterprise search stacks often move to rank fusion such as RRF because it is more stable across query classes and less sensitive to score calibration issues.

7. Anchor Boosting, Deep Research, Graph, And Temporal Layers

The repository includes several heuristic augmentation layers on top of the baseline retrievers. Query anchors are extracted from the question, from custom query text, and from optional manual anchor terms. Matching anchors in heading or snippet text add an anchor score to the combined score. The general anchor boost formula is approximately $\min(0.24, \text{matched} * 0.035 + \max(\text{matched} - 1, 0) * 0.025)$, then added directly to the current combined score.

Deep research expands the candidate search space and lowers the hybrid alpha to 0.2, making lexical evidence more influential. It can run supplemental keyword search using anchor terms, source-scoped rare-anchor scans, and neighbor expansion. Rare-anchor window candidates use a score formula capped at 0.36 based on matched anchors, lexical overlap, and a causal-term bonus. Neighbor candidates derive from source-local chunk adjacency and can be added with fixed or derived bonuses.

Graph and temporal retrieval are not separate full retrievers. They are additive layers on top of baseline candidates. Graph evidence can add $\text{graph_score} * 0.35$ to an existing result or create a supplemental candidate with $\text{graph_score} * 0.2$. Temporal metadata can contribute boosts such as 0.12 for exact normalized date matches, 0.08 for year matches, and 0.10 for version matches, blended into the combined score with a 0.2 weight.

Limitations and consequences

These layers are powerful for targeted recall rescue, but they are still heuristic and additive. They are best understood as query-time boosts or reranking aids, not as fully general metadata filtering or enterprise-grade multi-index retrieval orchestration.

8. Metadata Filtering

Explicit metadata filtering exists through SearchFilters: `source_type`, `source_id`, `source_part_id`, and `locator_filter`. These are applied at DB query time for vector and keyword search. That is real filtering, and it works consistently across retrieval modes that use those DB surfaces.

What is not implemented is broad inferred metadata filtering. If a query semantically suggests a year, version, or relationship without an explicit filter, the current system generally boosts likely matches through the router or through temporal and graph scoring. It does not silently construct a general metadata filter for all queries.

Limitations and consequences

This is a clean and safe design for a PoC because explicit filters are transparent. The tradeoff is that attribute-aware retrieval is limited. Enterprise systems often add inferred filters for dates, versions, entities, products, or customer-specific taxonomies once there is a robust evaluation loop to justify the extra complexity.

9. Reranking

Cross-encoder reranking is implemented in `backend/app/core_rag/reranker.py` and is optional. The active model default is `cross-encoder/ms-marco-MiniLM-L-6-v2`. When enabled, the system creates query-snippet pairs, truncates snippet length at 2000 characters, predicts rerank scores, and sorts candidates descending by rerank score before truncating to the requested top k.

This is a meaningful quality lever already present in the repo. It is currently disabled by default through `RERANK_ENABLED = false`. That default is sensible for local and low-latency development environments, but enterprise retrieval quality would usually benefit from measuring reranking cost and turning it on in quality-sensitive deployments.

Limitations and consequences

There is no MMR or diversity filter after reranking, no field-aware reranking, no budget-aware control of rerank depth, and no query-type-triggered rerank policy. The reranker is present and useful, but still just one optional stage.

10. Query Transformation

Full query transformation is absent. There is no implemented LLM query rewrite, no query expansion stage, and no HyDE. The closest substitutes are manual or heuristic controls: `custom_query`, `exact_phrase_bias`, anchor-term extraction, and the deep-research anchor query path.

These are helpful power-user or developer knobs, but they are not the same as a systematic transformation pipeline. There is also no evaluation framework specifically measuring when transformed queries beat raw queries.

Limitations and consequences

This is one of the clearest enterprise gaps. Query transformation is off by design because it adds latency, cost, and failure modes, but the codebase has enough surrounding structure that it could be added behind explicit config flags later.

11. Semantic Caching

A semantic caching layer is absent. The repository does use normal process-level object caching: the embedding model is cached as a singleton, and the reranker model is cached as a singleton. That improves startup and repeated-query latency within one process, but it is not semantic caching.

There is no query similarity cache, no retrieval result cache keyed by semantic neighborhood, no answer cache tied to corpus freshness, and no invalidation model tied to source reingestion or re-enrichment.

Limitations and consequences

This keeps correctness simple, but it leaves latency and infrastructure cost savings on the table. Semantic caching is a natural future feature, but it should remain off by default until invalidation rules are designed carefully.

12. Answer Grounding And Failure Semantics

The answering layer is relevant to retrieval analysis because a visible failure can happen after relevant chunks are retrieved. The current system validates citations, sanitizes unsafe citations, performs a second-pass grounded repair when the first answer is weak, and falls back to Not found in provided sources when a coherent grounded answer still cannot be formed.

This means some user-visible misses are retrieval misses, while others are answer-grounding misses. That distinction matters when diagnosing why hybrid appears to fail or why a lexical mode appears to win. The current debug traces are a good base, but they do not yet expose a single user-facing confidence model.

Limitations and consequences

The code is safer than many simple RAG demos, but it does not yet separate retrieval confidence from answer confidence in a product-facing way. That is another good future enterprise improvement.

13. Evaluation

The repository already has a meaningful offline evaluation framework. Retrieval evaluation is handled in `backend/app/eval/retrieval_eval.py`. Answer and compare evaluation are handled in `backend/app/eval/enriched_eval.py`. Cross-mode benchmark comparison is handled in `backend/app/eval/compare_eval.py`. Fixture files live under `backend/tests/fixtures/eval` and already distinguish retrieval cases, answer cases, compare cases, and benchmark cases.

This evaluation stack is good at controlled regression testing, source-coverage checks, citation correctness checks, fallback checks, and descriptive multi-mode benchmarking. It is not yet a real production feedback loop. There is no live user-query mining, no annotation workflow, no drift tracking, no online experimentation framework, and no acceptance metric tied to real user outcomes.

Limitations and consequences

The existing evaluation harness is strong for a PoC and for iterative engineering work. The enterprise gap is not the lack of any evaluation, but the lack of real-user-query capture, labeling, and continuous offline-online measurement.

14. Recommended Defaults And Roadmap

Keep and tune now: keep the default retrieval mode as hybrid, but describe it precisely as baseline hybrid with linear fusion rather than an enterprise-final hybrid. Keep `USE_QUERY_ROUTER` set to true. Keep graph and temporal layers opt-in and artifact-dependent. Keep type-specific chunking. Keep reranking available but disabled by default for local and developer-first environments until latency budgets are measured.

Add next: first add an RRF fusion option behind a configuration flag such as `HYBRID_FUSION_METHOD` with values linear and rrf. Second, expand router heuristics for quote lookup, final words, exact phrase retrieval, and code or SKU patterns. Third, add rerank policy controls: when reranking runs, how many candidates it sees, and whether reranking is tied to query class. Fourth, add semantic caching behind an explicit toggle. Fifth, add query transformation features such as rewrite, expansion, and HyDE, all disabled by default. Sixth, create a user-query-driven evaluation pipeline.

Longer-term enterprise upgrades: field-weighted keyword indexing, sparse plus dense retrieval, MMR or diversity filtering, inferred attribute-aware filtering, tokenizer-aware and layout-aware chunking, domain-specific embedding and reranking models, and a live feedback loop tied to online metrics.

15. Building Better Evaluation Sets From Real Queries

The next evaluation step should be based on real user behavior. Collect production-like queries, user retry paths, failure reports, and mode-switch behavior. Cluster queries by retrieval intent: exact lookup, semantic lookup, cross-file synthesis, relationship-heavy, temporal, quote or factoid, and scoped rescue. Convert a curated sample into fixture-style benchmark cases that store gold evidence and expected source or chunk support, not only expected prose answers.

Recommended metrics by stage: retrieval should track Recall@k, MRR, hit rate on expected source or chunk, and latency. Reranking should track top-1 improvement, rank lift, and nDCG or a close substitute against labeled relevance. Answering should track citation validity, groundedness, fallback rate, and unsupported-answer rate. Product-level measurement should track user retry rate, mode-switch rate, not-found rate, and accepted-answer rate.

Enterprise gap note: the current eval harness is already useful for controlled regression and benchmark comparison, but it is not yet a live feedback loop tied to real user behavior, annotation, or offline-online experimentation.

Functionality Matrix

Capability	Current Status	Where In Code	How It Works Today	Default Setting	Main Limitations	Recommended Next Step	Keep Off / Keep On / Add Toggle
Type-specific chunking	Present	chunking.py and adapters	Separate chunkers for pdf, docx, pptx, xlsx, eml, txt, md with shared word-window logic	On	Heuristic, not tokenizer-aware or query-adaptive	Keep and tune later with smarter chunk policies	Keep On
Dense embeddings	Present	embedder.py	Single normalized dense embedding model for all chunks and queries	BAAI/bge-smal l-en-v1.5	No sparse path, no multi-vector, no field-specific embeddings	Add model/config abstraction later	Keep On
Keyword retrieval	Present	repo_search.py	PostgreSQL FTS with tsquery fallback and soft-anchor fallback	On by mode/router	Not field-weighted, English-centric assumptions	Add field-weighted keyword indexing	Keep On
Vector retrieval	Present	repo_search.py	PGVector nearest-neighbor search over chunk embeddings	On by mode/router	Single dense space only	Keep and benchmark domain-specific models later	Keep On
Hybrid fusion	Present	retrieval.py	Linear blend: $0.65 * \text{vector_score} + 0.35 * \text{keyword_score}$	Default retrieval mode hybrid	Not RRF, truncation-sensitive	Add HYBRID_FUSION_METHOD with rrf option	Add Toggle
Query router	Present	query_router.py	Heuristic mode routing when mode is omitted	USE_QUERY_ROUTER=true	Exact-lookup detection too narrow	Expand quote, code, SKU, and final-words heuristics	Keep On
Deep research extras	Present	retrieval.py	Anchor extraction, supplemental keyword search, rare-anchor scans, neighbor expansion	Off unless requested	Heuristic-heavy and source-scope dependent	Keep as expert mode and evaluate gains	Keep Off by default
Metadata filtering	Partial	retrieval.py and repo_search.py	Explicit source/type/part/locator filters only	Manual when supplied	No broad inferred metadata filtering	Add attribute-aware filtering later	Keep On for explicit filters
Graph and temporal layers	Partial	retrieval.py and query_router.py	Query-time boosts and reranking using artifacts	Opt-in by mode and artifact	Not universal filtering; artifact-dependent	Keep opt-in and artifact-aware	Keep On selectively
Cross-encoder reranking	Present	reranker.py	Post-retrieval rerank with MiniLM cross-encoder	RERANK_ENABLED=false	No MMR, no policy controls, no field-aware rerank	Add rerank policy and depth controls	Add Toggle

Capability	Current Status	Where In Code	How It Works Today	Default Setting	Main Limitations	Recommended Next Step	Keep Off / Keep On / Add Toggle
Query rewrite, expansion, HyDE	Absent	Not implemented	Only partial substitutes: custom_query, exact_phrase_bias, anchors	Off	No systematic transform layer	Add disabled-by-default transform stack	Add Toggle
Semantic cache	Absent	Not implemented	Only model object caching, no semantic query-result cache	Off	No semantic reuse or invalidation policy	Add SEMANTIC_CACHE_ENABLED later	Add Toggle
Evaluation harnesses	Present	eval/*.py and fixture files	Fixture-based retrieval, answer, compare, and benchmark evaluation	Manual engineering workflow	No real-user loop	Keep and extend with user-query data	Keep On
Real user query mining and drift tracking	Absent	Not implemented	No production query capture, clustering, labeling, or drift analysis	Off	No live improvement loop	Add logging, clustering, annotation, and offline-online metrics	Add Toggle later